UCS 503 - Software Engineering

Real-Time Library Seat Occupancy Management System

An IoT-Based Live Seat Availability System

Submitted by:

1024031135 Saksham Kansal
1024031118 Ishaan Gupta
1024031137 Sanshi

Computer Engineering Department (COE)

Submitted to:

Ms. Jhonsy Bansal

August 2026

Computer Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 3
  - 1.2.1 Scope and Organisation of the Report . . . 4

2 Methodology and Design **5**
- 2.1 Requirements Analysis . . . 5
  - 2.1.1 Stakeholders . . . 5
  - 2.1.2 Functional Requirements . . . 5
  - 2.1.3 Non-Functional Requirements . . . 6
- 2.2 System Architecture . . . 6
  - 2.2.1 High-Level Design . . . 6
  - 2.2.2 Seating Layout . . . 7
  - 2.2.3 Hardware Design . . . 7
  - 2.2.4 Data Design . . . 8
- 2.3 UML and Process Models . . . 9
  - 2.3.1 Use Case Diagram . . . 9
  - 2.3.2 Data Flow Diagrams . . . 9
  - 2.3.3 Behavioural Models . . . 10
- 2.4 Implementation Details . . . 12
  - 2.4.1 Firmware (ESP32) . . . 12
  - 2.4.2 Cloud Layer and Web Application . . . 13

3 Results and Evaluation **14**
- 3.1 Testing Methodology . . . 14
- 3.2 Analysis . . . 14
  - 3.2.1 Limitations . . . 14

4 Conclusion **15**
- 4.1 Summary of Work . . . 15
- 4.2 Future Work and Recommendations . . . 15

Bibliography **16**

Chapter 1

Introduction

1.1 Background and Context

University libraries are among the most heavily used shared spaces on a campus, and demand peaks sharply around examinations and assignment deadlines. At such times students must physically walk through reading halls to discover where a seat is available. Conventional gate counters report how many people are inside the building, but not *where* the free seats are. The result is wasted search time, continuous hallway movement and noise that disturbs students who are already studying.

Low-cost Wi-Fi microcontrollers, proximity sensors and managed real-time cloud databases now make it feasible to monitor every seat and publish its state to any phone or display within a fraction of a second.

1.1.1 Problem Statement

The current way of locating a study seat has three shortcomings, summarised in Table 1.1.

Problem	**Description**	**Operational need**
Manual fragmentation	Students must physically navigate multiple floors and aisles during peak hours, causing hallway noise and wasted search time.	Remote, instant knowledge of free seats before walking to the hall.
Low transparency	Static headcount boards at library gates give lagging aggregate metrics without showing desk-level availability.	Seat-level visibility rather than a single building-wide count.
Administrative blindspot	Staff lack real-time and historical analytics to balance traffic across reading halls or to optimise HVAC and facility allocation.	Live and historical utilisation analytics for facility planning.

Table 1.1: Core problems and the corresponding operational need

1.2 Project Objectives

1. **Detection:** Detect physical presence at every monitored study seat using low-cost IR sensors.

2. **Noise filtering:** Filter sensor noise so that momentary obstructions do not change a seat's state.
3. **Transmission:** Transmit seat states to the cloud over Wi-Fi with minimal latency.
4. **Student view:** Present a live seat map and capacity metrics on mobile browsers and on an entrance kiosk display.
5. **Admin view:** Give administrators an analytics dashboard for utilisation, session time and peak-hour monitoring.

1.2.1 Scope and Organisation of the Report

The system covers sensing, communication, cloud synchronisation and presentation for study seats in a library zone. It identifies only whether a seat is occupied; it does not capture images, identify individuals or enforce seat reservations. The prototype demonstrates the complete data path from a sensor to a browser, and the design scales to additional seats and zones by adding sensor nodes and database branches.

Chapter 2 covers the requirements, architecture, UML models and implementation; Chapter 3 gives the test plan, discussion and limitations; and Chapter 4 concludes with future scope.

Chapter 2

Methodology and Design

2.1 Requirements Analysis

2.1.1 Stakeholders

- **Students:** Need to know which seats are free and how full the library is before and while travelling to it.
- **Library staff / administrators:** Need live occupancy and historical analytics for traffic balancing and facility allocation.
- **Sensor node (system actor):** Continuously senses seats and publishes state.

2.1.2 Functional Requirements

ID	Requirement
FR1	Detect the presence of a person at each monitored seat using an active IR sensor.
FR2	Apply a 3-second software debounce window before a seat changes state.
FR3	The ESP32 node encodes binary seat states into a JSON payload and sends it over Wi-Fi to the cloud database.
FR4	The cloud database keeps a synchronised state tree and notifies clients of changes without manual refresh.
FR5	The student dashboard shows a floor map: vacant seats green, occupied seats red.
FR6	The dashboards show total, occupied and available seats and the occupancy rate.
FR7	The entrance kiosk displays live capacity and availability for the zone.
FR8	The administrator dashboard shows average session time, peak hour, weekly trend, hourly peak distribution and the occupied/available split.

Table 2.1: Functional requirements

2.1.3 Non-Functional Requirements

ID	Attribute	Requirement
NFR1	Performance	Sub-second cloud-to-client synchronisation (excluding the deliberate 3-s debounce).
NFR2	Reliability	Momentary obstructions must not cause flickering of seat colours.
NFR3	Usability	The student interface is responsive and usable on a mobile browser.
NFR4	Scalability	Seats and zones can be added without redesigning the data model.
NFR5	Cost	Inexpensive off-the-shelf sensing hardware.
NFR6	Privacy	No images or personal identifiers; only a binary presence state is recorded.

Table 2.2: Non-functional requirements

2.2 System Architecture

2.2.1 High-Level Design

The system is organised in three layers, shown in Figure 2.1: an *edge layer* that senses and encodes seat state, a *cloud layer* that stores and distributes it, and a *presentation layer* that renders it for students and staff.

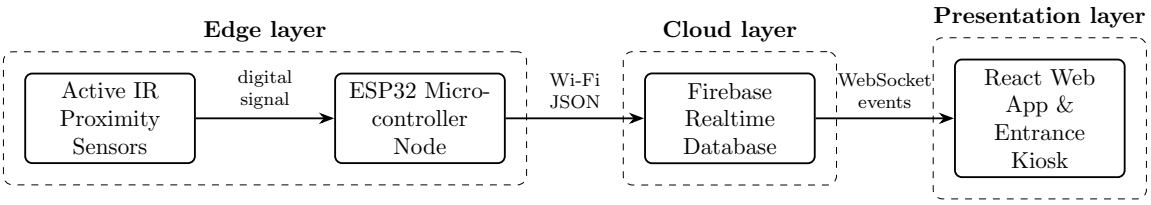


Figure 2.1: End-to-end system architecture

Active IR sensors on the seats feed an ESP32 node [Esp], which publishes debounced seat states as JSON over Wi-Fi to a Firebase Realtime Database [Goo]; the database pushes WebSocket events to the React web application and entrance kiosk [Met].

2.2.2 Seating Layout

Figure 2.2 shows the physical arrangement. Each seat has an IR sensor mounted at the table edge facing the chair, wired to a central controller that connects to an ESP32 Wi-Fi node. Here a vacant seat is drawn white and an occupied seat dark; the web dashboards show green and red.

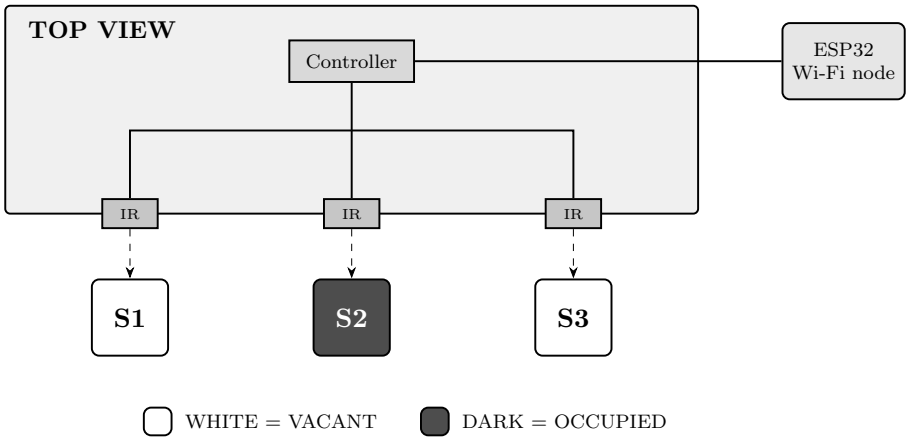


Figure 2.2: Seating layout (top view): per-seat IR sensors wired to a controller and an ESP32 node

2.2.3 Hardware Design

Component	**Role**
ESP32 development board	Reads the GPIO inputs, runs the debounce logic, builds JSON payloads and connects to the library Wi-Fi [Esp].
Active IR proximity sensors	One per seat; output a digital presence signal.
Push buttons (two)	Auxiliary manual inputs on GPIO 18 and GPIO 19, returning to ground.
5 V power supply	Powers the ESP32; an optional DC–DC step-down module can regulate the input.
Perforated board, jumper wires	Mechanical and electrical assembly of the prototype.

Table 2.3: Hardware components of the sensing node

Circuit layout. The schematic in Figure 2.3 shows the reference wiring. The 5 V supply and ground feed the ESP32. IR Sensor 1 and IR Sensor 2 connect to GPIO 27

and GPIO 28, and two push buttons connect to GPIO 18 and GPIO 19 and return to ground.

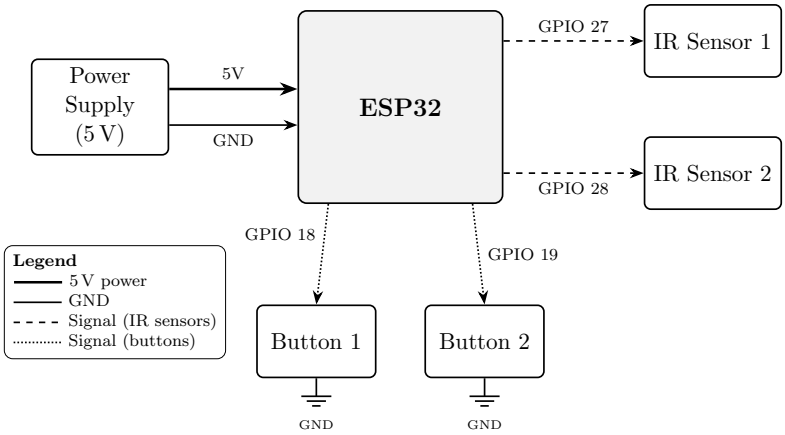


Figure 2.3: Circuit layout of the ESP32 sensing node

2.2.4 Data Design

The Firebase Realtime Database stores state as a JSON tree [Goo]. Each seat is a leaf holding a boolean occupancy flag. Because every client subscribes to the same tree, a single write by the ESP32 reaches all connected dashboards. Figure 2.4 shows the schema and Listing 2.1 the corresponding payload.

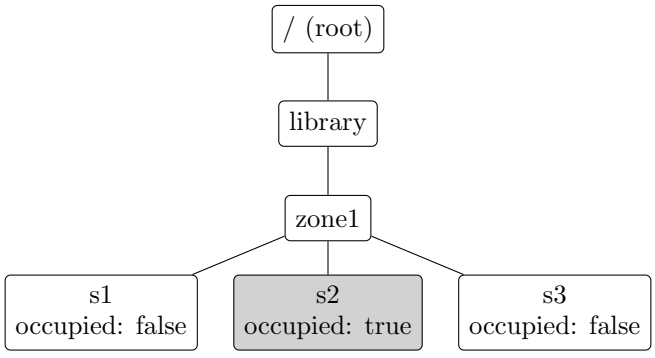


Figure 2.4: JSON state-tree schema (S2 occupied, S1 and S3 vacant)

Listing 2.1: Illustrative seat-state payload

```
{ "library": { "zone1": {
    "s1": { "occupied": false },
    "s2": { "occupied": true  },
    "s3": { "occupied": false } } } }
```

2.3 UML and Process Models

2.3.1 Use Case Diagram

Figure 2.5 shows the actors and their interactions with the system. Students use the live seat map and entrance availability views, administrators use the analytics views, and the sensor node detects presence and publishes state [BRJ05].

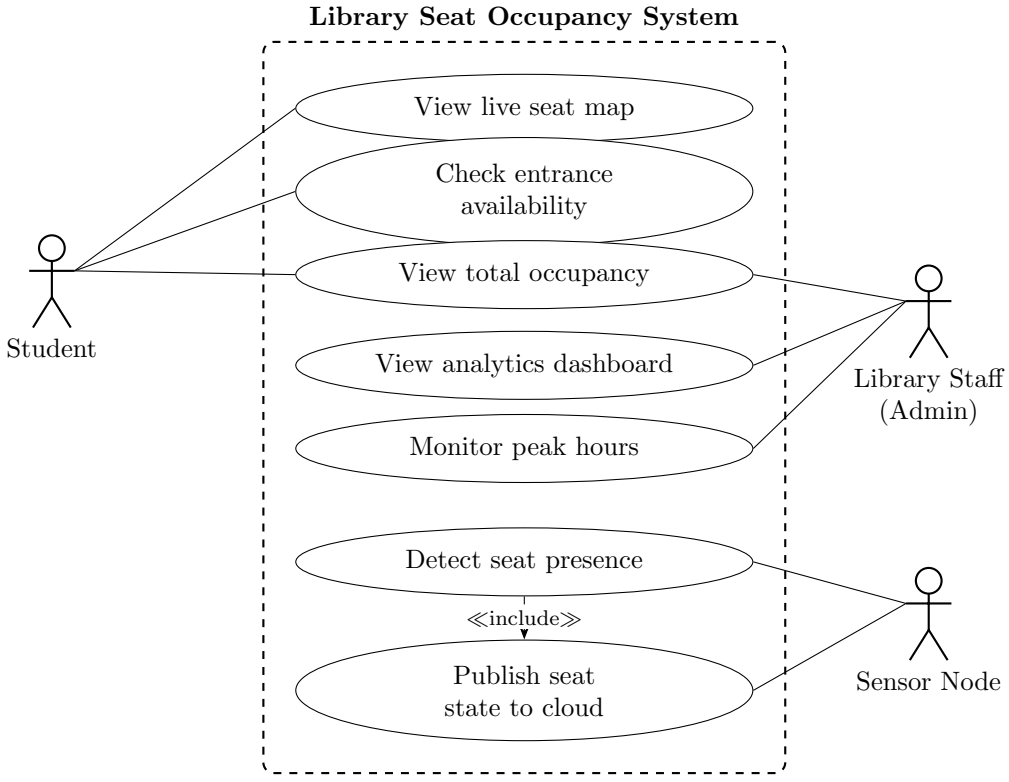


Figure 2.5: Use case diagram

2.3.2 Data Flow Diagrams

The context-level diagram (Figure 2.6) treats the whole system as one process, and the level-1 diagram (Figure 2.7) decomposes it into four processes and one data store.

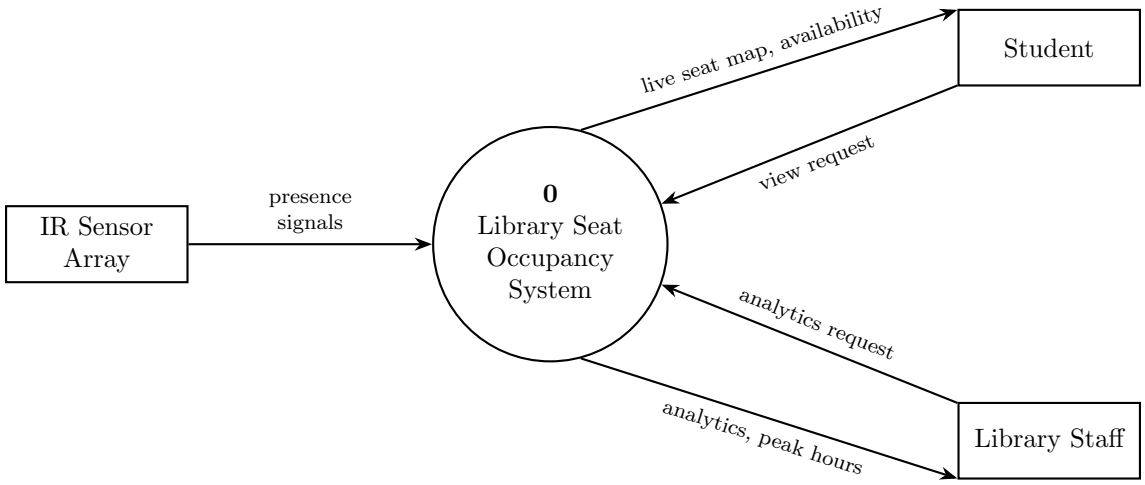


Figure 2.6: Data flow diagram – level 0 (context diagram)

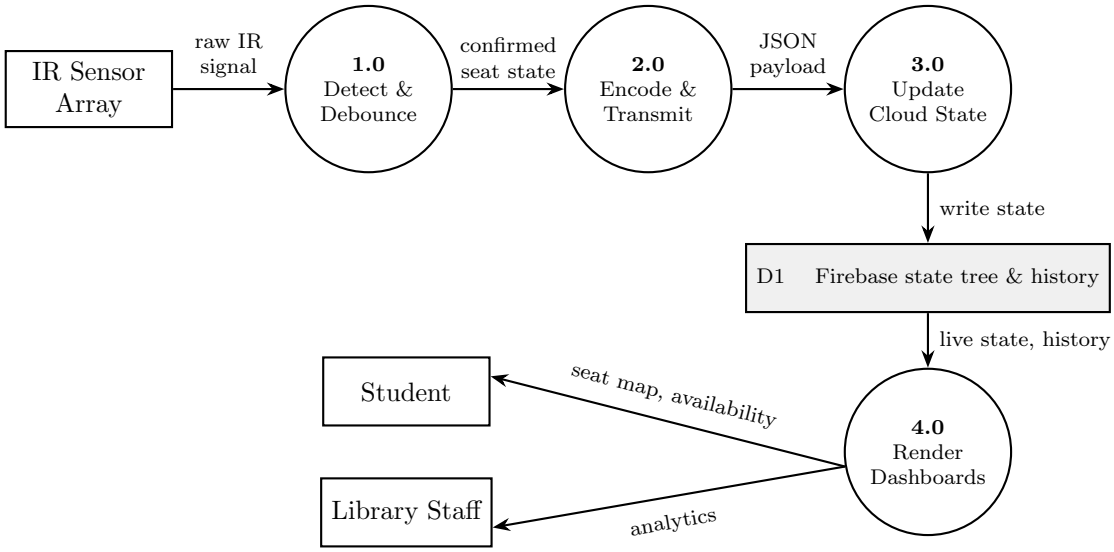


Figure 2.7: Data flow diagram – level 1

2.3.3 Behavioural Models

Sequence diagram. Figure 2.8 follows one seat-occupancy event from a student sitting down to the seat turning red on every connected dashboard.

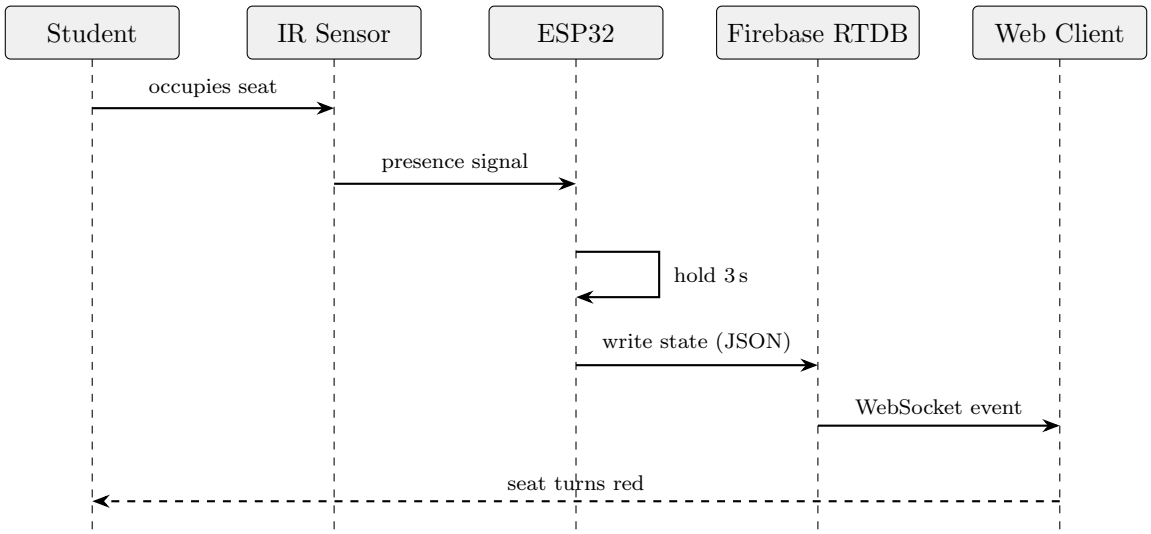
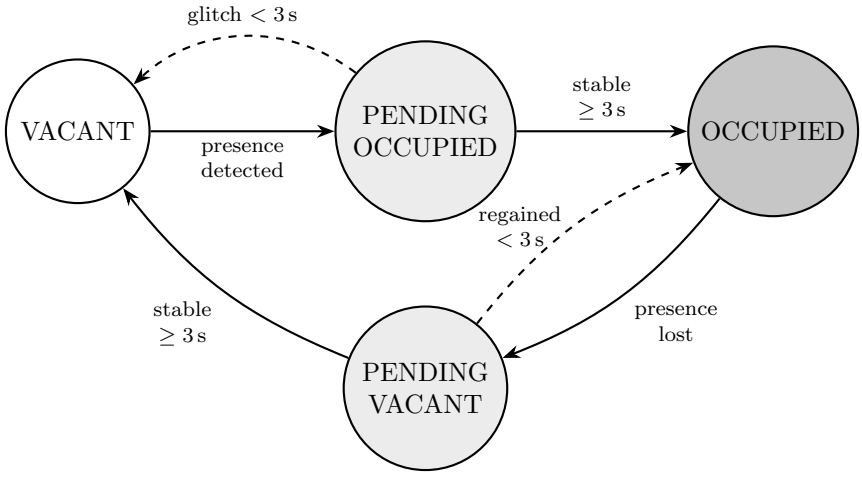


Figure 2.8: Sequence diagram for a seat-occupancy event

Seat state machine. Each seat is modelled as the four-state machine in Figure 2.9. A seat changes between VACANT and OCCUPIED only after the new reading has been stable for 3 seconds; a shorter disturbance returns it to the previous confirmed state (dashed transitions).



Dashed arrows: transient disturbance filtered by the debounce window.

Figure 2.9: Seat state machine with 3-second debounce

Firmware flowchart. The main loop of the ESP32 firmware is shown in Figure 2.10.

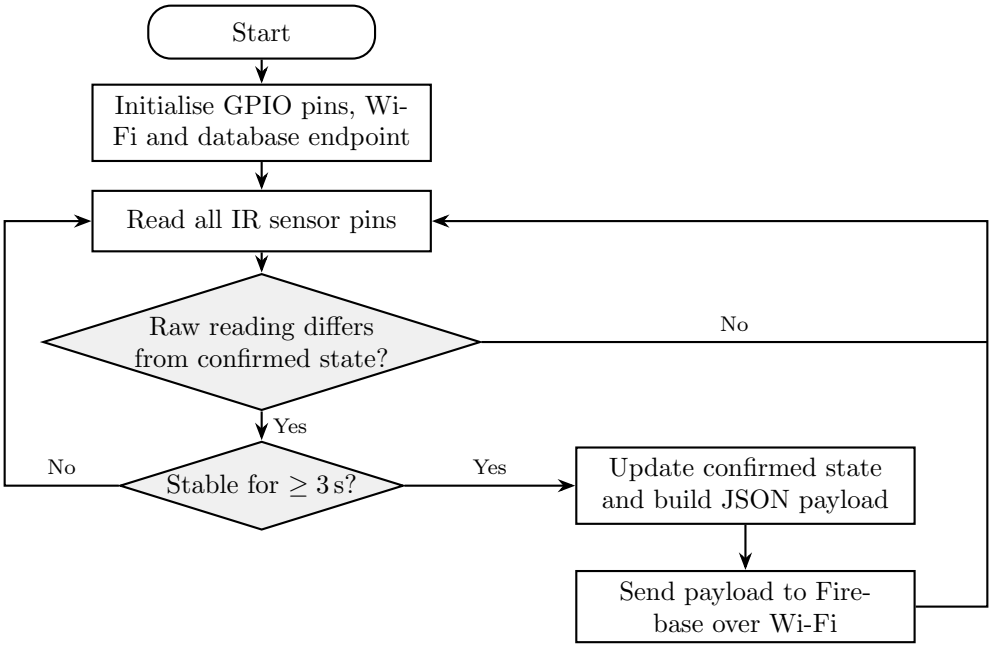


Figure 2.10: Firmware main-loop flowchart

2.4 Implementation Details

2.4.1 Firmware (ESP32)

The firmware polls the IR pins, applies the 3-second debounce per seat and publishes the confirmed states as JSON. Listing 2.2 is an illustrative implementation of this logic; pin numbers follow Figure 2.3.

Listing 2.2: Illustrative firmware logic for debounce and publishing

```
constexpr uint32_t DEBOUNCE_MS = 3000;   // 3-second debounce window
struct Seat { uint8_t pin; bool confirmed, candidate; uint32_t changedAt; };
Seat seats[2] = { {27, false, false, 0}, {28, false, false, 0} };

void publishState() {                     // build JSON and write once
  String json = "{";
  for (int i = 0; i < 2; ++i) {
    json += "\"s" + String(i + 1) + "\":{\"occupied\":";
    json += seats[i].confirmed ? "true}" : "false}";
    if (i < 1) json += ",";
  }
  json += "}";
  HTTPClient http;  http.begin(DB_URL);
  http.PUT(json);   http.end();           // clients notified by the database
}
void loop() {
  bool dirty = false;
  for (auto &s : seats) {
    bool raw = (digitalRead(s.pin) == LOW);         // active-low IR module
    if (raw != s.candidate) { s.candidate = raw; s.changedAt = millis(); }
    if (s.candidate != s.confirmed && millis() - s.changedAt >= DEBOUNCE_MS) {
      s.confirmed = s.candidate;  dirty = true;
    }
  }
  if (dirty) publishState();
}
```

2.4.2 Cloud Layer and Web Application

The Firebase Realtime Database holds the state tree of Figure 2.4. Clients attach listeners to the tree and receive updates as events over a persistent WebSocket connection, so no polling is required.

The web application is built with React [Met] and branded *SmartLib*. It has a *Student* view and an *Admin* view, reachable from the navigation bar. The floor map is made of SVG nodes whose fill colour is bound to each seat's occupancy flag, so a database update re-renders the affected seat immediately.

Student dashboard. Shows live capacity metrics and the seat layout. Summary cards report total, occupied and available seats and the occupancy rate (in the example shown: 4 seats, 2 occupied, 2 available, 50%). A seat-layout panel shows seats S01–S04 around a study table; available seats are green and occupied seats are red.

Admin dashboard. Provides utilisation analytics. Summary cards report total seats, occupied seats, average session time and the peak hour. A weekly occupancy trend, an hourly peak-hours histogram and a donut chart of the available/occupied split support traffic balancing and facility planning.

Entrance kiosk. The same application shows live capacity at the library gate.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The planned test cases in Table 3.1 verify the functional and non-functional requirements of Chapter 2.

ID	Scenario	Expected outcome	Req.
T1	A person occupies a vacant seat	After the 3-s debounce the seat turns red on all dashboards	FR1, FR2, FR5
T2	The person leaves the seat	After 3 s the seat turns green	FR1, FR2, FR5
T3	A hand or bag passes the sensor for less than 3 s	No state change is published	FR2, NFR2
T4	Two clients are open simultaneously	Both update from the same database event without refresh	FR4
T5	Occupancy counts are inspected	Occupied + available equals total seats; the rate is correct	FR6
T6	The Wi-Fi connection drops and returns	The node resumes publishing and dashboards show the current state	NFR1
T7	The kiosk page is opened at the gate	Live capacity matches the student dashboard	FR7
T8	Admin dashboard is opened	Session time, peak hour and charts are displayed	FR8

Table 3.1: Test cases and expected outcomes

3.2 Analysis

The prototype demonstrates the complete data path: an IR sensor feeding an ESP32 node, a JSON state in a cloud database, and browser dashboards updated through WebSocket events. Seat-level sensing removes the main weakness of gate counters, and the 3-second debounce keeps the colour indicators stable.

3.2.1 Limitations

An IR sensor detects an obstruction, not a person, so an object left on a desk can be reported as occupied. The system depends on library Wi-Fi and the cloud service, and larger deployments need more nodes, pin and power planning and sensor calibration.

Chapter 4

Conclusion

4.1 Summary of Work

This project delivers a working architecture for a real-time, seat-level library occupancy system built from low-cost IoT hardware and a reactive cloud stack. It addresses the three identified problems: it reduces manual searching by showing free seats remotely, replaces aggregate gate counts with micro-level desk availability, and gives administrators live and historical analytics.

4.2 Future Work and Recommendations

1. Combine IR with a second sensing modality to distinguish a person from an object.
2. Add seat reservation and time-limited holds.
3. Forecast occupancy from the stored history to predict peak hours.
4. Adopt a lightweight messaging protocol such as MQTT and battery or low-power operation.
5. Extend to multi-zone and multi-floor deployment with a campus-wide map.
6. Send push notifications when a preferred seat or zone becomes free.

Bibliography

[BRJ05] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. 2nd ed. Addison-Wesley, 2005.

[Esp] Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems. `https://www.espressif.com`.

[Goo] Google. *Firebase Realtime Database Documentation*. `https://firebase.google.com/docs/database`.

[Met] Meta Open Source. *React Documentation*. `https://react.dev`.

[PM19] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th ed. McGraw-Hill, 2019.

[Som16] Ian Sommerville. *Software Engineering*. 10th ed. Pearson, 2016.